PROGRAMARE FUNCȚIONALĂ

Facultatea de Matematică și Informatică
Universitatea din București
Informatică ID, anul II

Cuprins

1 Administrativ **3**

2 De ce programare funcțională? **4**
2.1 4
2.2 Proprietăți 4
2.3 Haskell in industrie 6

3 Elemente de bază **7**
3.1 Instalare Haskell 7
3.2 Interpretor online Haskell 9
3.3 Sintaxă 9
3.4 Tipuri de date. Sistemul tipurilor 13
3.5 Funcții în Haskell. Terminologie 15
3.6 Fișiere sursă 17
3.7 Operatori 18
3.8 Exerciții 20
3.9 Secțiuni ("operator sections") 22

4 Recursivitate **25**

5 Funcții anonime (Lambda Expresii) **26**

6 Compunerea funcțiilor — operatorul . **27**

7 Liste **28**
7.1 Recursivitate pe Liste 32
7.2 Liste definite prin selecție (descrieri de liste) 33
7.3 Prelucrarea listelor folosind funcțiile: Map, Filter, Zip și Fold 36
7.3.1 Map 36
7.3.2 Filter 37
7.3.3 Zip 38
7.3.4 Fold 40

8 Tipuri de date algebrice **45**
8.1 Definire tipuri de date 45
8.2 Instanțe ale claselor de tipuri 51
8.3 Parțialitate - tipul Maybe 55
8.4 Variante - tipul Either 56

1 Administrativ

Instructor: Ana Iova, ana.iova@fmi.unibuc.ro

Programarea cursului în timpul semestrului:

Curs+Laborator 1 ..... 10.10.2026, 14.00-18.00, online
Curs+Laborator 2 ..... 11.10.2026, 14.00-18.00, online
Curs+Laborator 3 ..... 27.10.2026, 16.00-20.00, online
Curs+Laborator 4 ..... 10.11.2026, 16.00-20.00, online
Curs+Laborator 5 ..... 22.11.2026, 14.00-18.00, online
Curs+Laborator 6 ..... 29.11.2026, 14.00-18.00, online
Curs+Laborator 7 ..... 19.01.2027, 16.00-20.00, online

Obiectivele cursului: Cursul are scopul de a pune baza programării în Haskell.

Cursurile vor acoperi următoarea materie:

- elemente de bază de limbaj;
- tipuri de date, funcții;
- liste;
- funcții de nivel înalt;
- tipuri de date algebrice și clase de tipuri.

Notare: Examenul va consta într-un set de întrebări grilă din toată materia și exerciții de programare asemănătoare cu cele făcute în timpul semestrului.

Examinarea se va face pe calculatoarele din facultate.

La examen este permis ca material ajutător doar suportul de curs sub format pdf care este oferit de către profesor prin intermediul platformei Teams. Nu este permisă folosirea altor materiale online/fizice. Mai multe detalii vor fi oferite înainte de examinare.

Resurse suplimentare

- Pagina Haskell: `http://haskell.org`
- Hoogle `https://www.haskell.org/hoogle`
- Haskell Wiki `http://wiki.haskell.org`
- Cartea online "Learn You a Haskell for Great Good" `https://learnyouahaskell.github.io/`

2 De ce programare funcțională?

2.1 Programare Imperativă vs. Declarativă

Imperativă (*the How*)

- Îi dai calculatorului o listă clară de pași.
- Gândește-te la un **checklist** sau la o **rețetă**: faci asta, apoi asta, apoi asta.
- Programare procedurală / orientată pe obiecte.

Declarativă (*the What*)

- Îi spui doar ce vrei sa obții, nu fiecare pas de urmat.
- Ca la restaurant: tu spui doar "vreau paste"; bucătarul decide cum gătește pastele.
- Programare logică / baze de date / funcțională

Putem implementa programare funcțională și în alte limbaje de programare:

- Java 8, C++11, Python, JavaScript, ...
- Funcții anonime (λ-abstracții)
- Funcții de procesare a fluxurilor de date: filter, map, reduce

Exemplu: Ciurul lui Eratostene

```
primes = sieve [2..]
sieve (p:ps) = p : sieve [ x | x <- ps, mod x p /= 0 ]
```

2.2 Proprietăți ale acestui limbaj:

- **Haskell conține concepte de programare funcțională pură.**
  - **Funcțiile sunt valori.**
    - În loc să modificăm datele existente, calculăm valori noi din valorile existente, folosind funcții.
  - **Funcțiile sunt pure:** aceleași rezultate pentru aceleași intrări.
    - O bucată de cod nu poate corupe datele altei bucăți de cod.
    - Există o distincție clară între părțile pure și cele care comunică cu mediul extern.

- **Oferă suport pentru paralelism și concurență.**
- **Haskell este un limbaj elegant**
  - Idei abstracte din matematică devin instrumente puternice în practică
    - recursivitate, compunerea de funcții, functori, monade
    - folosirea lor permite scrierea de cod compact și modular
  - Rigurozitate: ne forțează să gândim mai mult înainte, dar ne ajută să scriem cod mai corect și mai curat
  - Curbă de învățare în trepte
    - Putem scrie programe mici destul de repede
    - Expertiza în Haskell necesită multă gândire și practică
    - Descoperirea unei lumi noi poate fi un drum distractiv și provocator `http://wiki.haskell.org/Humor`
- **Haskell e leneș** *(lazy)***:** orice calcul e amânat cât de mult posibil
  - Schimbă modul de concepere al programelor
  - Permite lucrul cu colecții potențial infinite de date precum [1..]
  - Evaluarea lenesă poate fi exploatată pentru a reduce timpul de calcul fără a denatura codul

    ```
    firstK k = take k primes
    ```

- **Haskell e minimalist:** mai puțin cod, în mai puțin timp, și cu mai puține defecte ... rezolvând totuși problema.

```
numbers = [1,2,3,4,5]
total = foldl (*) 1 numbers
doubled = map (* 2) numbers
```

Exemplu

Quicksort în Haskell

```
qsort :: Ord a => [a] -> [a]
qsort []     = []
qsort (p:xs) = (qsort $ filter (< p) xs ) ++ [p] ++ (qsort $
   filter (>= p) xs)
```

Quicksort în C++

```
void QuickSort(int v[], int st, int dr)
{
        if(st < dr)
        {
```

```
                int m = (st + dr) / 2;
                int aux = v[st];
                v[st] = v[m];
                v[m] = aux;
                int i = st , j = dr, d = 0;
                while(i < j)
                {
                if(v[i] > v[j])
                        {
                                aux = v[i];
                                v[i] = v[j];
                                v[j] = aux;
                                d = 1 - d;
                        }
                        i += d;
                        j -= 1 - d;
                }
                QuickSort(v, st , i - 1);
                QuickSort(v, i + 1 , dr);
        }
}
```

2.3 Haskell in industrie

Programarea funcțională e din ce în ce mai importantă în industrie

- Haskell e folosit în proiecte de Meta, Google, Microsoft etc.
- Alte proiecte mari scrise in Haskell:
  - `https://typeable.io/`
  - `https://serokell.io/`
  - `https://xmonad.org/`
  - `https://jaspervdj.be/hakyll/`



Figure 1: 10 motive pentru a folosi Haskell

Mai multe detalii la:

- Haskell in industrie - Wiki
- Serokell Blog Post: Best Haskell open source projects
- Typeable Blog Post: 7 Useful Tools Written in Haskell
- Serokell Blog Post: Why Fintech Companies Use Haskell
- Wasp Blog Post: How to get started with Haskell in 2022 (the straightforward way)

3 Elemente de bază

3.1 Instalare Haskell

Pentru a instala Haskell puteți folosi GHCup, urmărind instrucțiunile de aici [Instalare]

Deschideți un terminal si introduceți comanda `ghci` (în Windows este posibil să aveți instalat WinGHCi). După câteva informații despre versiunea instalată va apare
ghci> sau **Prelude**> (în funcție de versiunea instalată)

`ghci` este librăria standard: `https://hackage.haskell.org/package/base-4.17.0.0/docs/ghci.html`

Primele expresii în GHCi Înainte de a defini funcții și variabile, putem evalua expresii simple.

În interpretor puteți:

- să introduceți expresii, care vor fi evaluate atunci când este posibil:

```
ghci> 2+3
 5
ghci> 10 `div` 3
        3
ghci> False || True
 True
ghci> x
 <interactive>:10:1: error: Variable not in scope: x
ghci> x=3
ghci> x
 3
ghci> y=x+1
ghci> y
 4
ghci> head [1,2,3]
 1
ghci> head "abcd"
 'a'
ghci> tail "abcd"
 "bcd"
```

Funcțiile `head` și `tail` aparțin modulului standard **ghci**.

- să introduceți comenzi, orice comandă fiind precedată de ":".

Comenzi utile în GHCi

:t expr - Afișează tipul unei expresii.

:i nume - Afișează informații despre o funcție, un tip sau o clasă.

:l fisier.hs - Încarcă un fișier sursă.

:r - Reîncarcă fișierul curent.

:q - Închide interpretorul.

:cd director - Schimbă directorul curent.

```
ghci> :t True
True :: Bool

ghci> :t head
head :: [a] -> a

ghci> :t tail
tail :: [a] -> [a]
```

Exercițiul 3.1 Testați în interpretor următoarele comenzi/expresii.

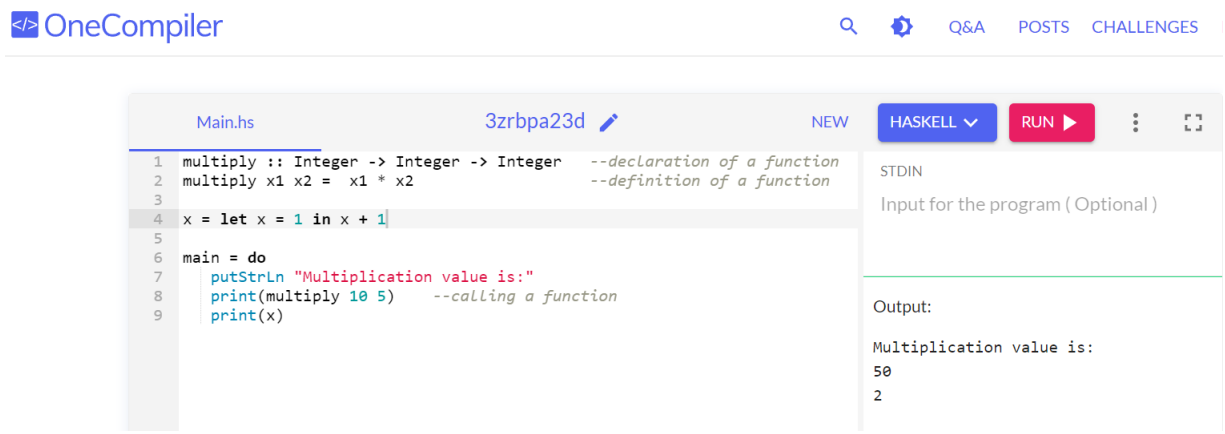
```
:?              even 3
:t True         15 `div` 2
:t "abc"        True || False
:t 'a'          10 /= 5
:t (&&)         length [1,2,3]
:t even
:i Bool
:i even
```

Citiți mai mult despre GHCi:
`https://downloads.haskell.org/~ghc/latest/docs/html/users_guide/ghci.html`

3.2 Interpretor online Haskell

Puteți folosi orice interpretor online de Haskell pentru a lucra probleme simple.

- `https://replit.com/~`
- `https://www.jdoodle.com/execute-haskell-online/`
- `https://onecompiler.com/haskell`



3.3 Sintaxă

Comentarii

```
-- comentariu pe o linie
{- comentariu pe
 mai multe
 linii -}
```

Identificatori

- șiruri formate din litere, cifre, caracterele _ și ' (apostrof)
- identificatorii pentru variabile încep cu literă mică sau _
- identificatorii pentru tipuri și constructori încep cu literă mare
- Haskell este sensibil la majuscule (case sensitive)

```
double x = 2 * x
data Point a = Pt a a
```

Blocuri și indentare.

Blocurile sunt delimitate prin indentare.

```
fact n = if n == 0
            then 1
            else  n * fact (n-1)

trei = let
          a = 1
          b = 2
       in a + b
```

Echivalent, putem scrie

```
trei  =  let {a = 1; b = 2} in a + b
trei  =  let a = 1; b = 2 in a + b
```

Variabile

Presupunem că fisierul `"elem_baza.hs"` conține

```
x=1
x=2
```

Ce valoare are x?

```
Prelude> :l elem_baza.hs
[1 of 1] Compiling Main             ( elem_baza.hs, interpreted )

elem_baza.hs:2:1: error:
    Multiple declarations of 'x'
    Declared at: elem_baza.hs:1:1
                 elem_baza.hs:2:1
  |
2 | x=2
  | ^
Failed, no modules loaded.
```

În Haskell, variabilele sunt `imutabile` *(immutable)* , adică: `=` **nu** este operator de atribuire, deci `x = 1` reprezintă o `legatură` *(binding)* (din momentul în care o variabilă este legată la o valoare, acea valoare nu mai poate fi schimbată).

`let .. in ...` este o `expresie` care crează un domeniu de vizibilitate local (permite declararea de variabile și funcții locale).

Presupunem că fișierul `testlet.hs` conține

```
x=1
z= let x=3 in x
```

```
Prelude> :l testlet.hs
[1 of 1] Compiling Main             ( testlet.hs, interpreted )
Ok, one module loaded.
*Main> z
3
*Main> x
1
*Main>
```

Alte exemple:

```
x = let
      z = 5
      g u = z + u
      in  let
            z = 7
          in g 0 + z
```

Ce valoare are x?

-- *x=12*

```
x = let z = 5; g u = z + u in let z = 7 in g 0
```

Ce valoare are x?

-- *x=5*

Clauza `... where ...` creaza scop local (permite declararea de variabile și funcții locale).

```
f x = g x + g x + z
    where
      g x = 2*x
      z =  x-1
```

let .. in ... este o expresie

```
x = [let y = 8 in y, 9]   -- x=[8,9]
```

where este o clauză disponibilă doar la nivel de definiție

```
x = [y where y = 8, 9]      -- error: parse error ...
```

Variabile pot fi legate și prin *pattern matching* la definirea unei funcții sau expresii **case**.

```
h x
  | x == 0     = 0
  | x == 1     = y + 1
  | x == 2     = y * y
  | otherwise = y
      where y = x*x
```

```
f x = case x of
        0 -> 0
        1 -> y + 1
        2 -> y * y
        _ -> y
          where y = x*x
```

Să definim funcția `maxim`

```
maxim :: Integer -> Integer -> Integer
maxim x y = if (x > y) then x  else y
```

Varianta cu indentare este:

```
maxim :: Integer -> Integer -> Integer
maxim x y =
    if (x > y)
        then x
        else y
```

Dorim acum să scriem o funcție care calculează maximul a trei numere. Evident, o varianta este

```
maxim3 x y z = maxim x (maxim y z)
```

Scrieți funcția `maxim3` fără a folosi `maxim`, utilizând direct `if` și scrierea indentată.

Putem scrie funcția `maxim3` folosind expresia `let...in` astfel

```
maxim3 x y z = let u = (maxim x y) in (maxim u z)
```

Atenție! Expresia `let...in` crează scop local.

Varianta cu indentare este

```
maxim3 x y z =
    let
        u = maxim x y
    in
        maxim u z
```

Exercițiul 3.2

- Scrieți o funcție `maxim4` folosind varianta cu `let..in..` și indentare.
- Modificați următoarele funcții astfel încât să folosească `let..in..` în loc de `where`.

```
fahrenheitToCelsius f = c
  where c = (f - 32) * 5 / 9

bmi2 greutate inaltime = greutate / h2
  where h2 = inaltime ^ 2
```

- Modificați următoarea funcție astfel încât să folosească `where` în loc de `let..in...`

```
notaFinala examen proiect =
  let media = 0.7 * examen + 0.3 * proiect
  in if media >= 5
      then "promovat" else "respins"
```

Citiți mai multe despre indentare: `https://en.wikibooks.org/wiki/Haskell/Indentation`

3.4 Tipuri de date. Sistemul tipurilor

Sistemul tipurilor de date are urmatoarele proprietăți:

- `tare` – garantează absența anumitor erori
- `static` – tipul fiecărei valori este calculat la compilare
- `dedus automat` – compilatorul deduce automat tipul fiecărei expresii

```
ghci> :t [('a',1,"abc")]
[('a',1,"abc")] :: Num b => [(Char, b, [Char])]
```

Variabile de tip În Haskell putem scrie funcții generice, care funcționează pentru mai multe tipuri de date.

Exemplu:

```
ghci> :t head
head :: [a] -> a
```

Litera `a` reprezintă o variabilă de tip. Aceasta înseamnă că funcția `head` poate primi o listă de orice tip și va întoarce un element de același tip. De exemplu:

```
ghci> head [1,2,3]
1
ghci> head "abc"
'a'
```

Exemple de tipuri de date:

- `Tipurile de bază`: Int, Integer, Float, Double, Bool, Char, String
- `Tipuri compuse`: tupluri (secvențe de tipuri deja existente) si liste

```
ghci> :t ('a', True)
('a', True) :: (Char, Bool)

ghci> :t (1 :: Int, 'a', "ab")
(1 :: Int, 'a', "ab") :: (Int, Char, [Char])

ghci> fst (1,'a') -- numai pentru perechi
ghci> snd (1,'a')
```

```
ghci> :t ["ana", "ion"]
["ana", "ion"] :: [[Char]]

ghci>:t [True, False, True]
[True, False, True] :: [Bool]
```

- `Tipuri noi definite de utilizator:`

```
data RGB = Rosu | Verde | Albastru
data Point a = Pt a a     -- tip parametrizat
-- a este variabila de tip
```

- `Integer`: 4, 0, -5
- `Float`: 3.14
- `Char`: 'a','A', '\n'
- `Bool`: True, False
- `String`: "prog\ndec"

```
type String = [Char] -- sinonim pentru tip
```

Ce răspuns primim în GHCi dacă introducem comanda?

```
ghci> :t 1
```

Răspunsul primit este:

```
1 :: Num a => a
```

Semnificația este următoarea:

- `a` este un *parametru de tip*
- `Num` este o clasă de tipuri (Va fi discutat mai tarziu)
- `1` este o valoare de tipul `a` din clasa `Num`

```
ghci> :t [1,2,3]
[1,2,3] :: Num t => [t]
```

3.5 Funcții în Haskell. Terminologie

În Haskell funcțiile sunt formate din mai multe componente:

- prototipul funcției: format din numele funcției, tipurile de date ale parametrilor și tipul de date rezultat.
- definiția funcției: formată din numele funcției, parametri formali și corpul funcției (expresia care dă rezultatul)
- aplicarea funcției: numele funcției și parametri actuali

Exemplu: funcție cu un argument

- **Prototipul funcției** double :: **Integer** −> **Integer**
  - numele funcției: `double`
  - signatura funcției: `Integer -> Integer`
- **Definiția funcției** double **elem** = **elem** + **elem**
  - numele funcției: `double`
  - parametrul formal: `elem`
  - corpul funcției: `elem + elem`
- **Aplicarea funcției** double 5
  - numele funcției: `double`
  - parametrul actual (argumentul): `5`

Exemplu: funcție cu două argumente

- **Prototipul funcției** add:: **Integer** −> **Integer** −> **Integer**
  - numele funcției: `add`
  - signatura funcției: `Integer -> Integer -> Integer`
- **Definiția funcției** add elem1 elem2 = elem1 + elem2
  - numele funcției: `add`
  - parametrii formali: `elem1, elem2`
  - corpul funcției: `elem1 + elem2`
- **Aplicarea funcției** add 3 7
  - numele funcției: `add`

– argumentele: 3, 7

Exemplu: funcție cu un argument de tip tuplu

```
dist :: (Integer, Integer) -> Integer
dist (x,y) = abs(x-y)
-- apel: dist(2,4)
```

Tipuri de funcții

Putem da tipul unei funcții explicit, ca în exemplele de mai sus, sau putem omite linia respectivă caz în care interpretorul găsește automat signatura funcției, în funcție de implementare, funcțiile și operațiile folosite de noi. Aceasta poate să nu fie exactă cum am fi dat-o noi.

Ne reamintim că în Haskell există o comandă `:t` care afisează tipul unei funcții.

Exemple:

```
ghci> :t abs
abs :: Num a => a -> a
ghci> :t div
div :: Integral a => a -> a -> a

ghci> :t (:)
(:) :: a -> [a] -> [a]
ghci> :t (++)
(++) :: [a] -> [a] -> [a]

ghci> :t zip
zip :: [a] -> [b] -> [(a, b)]
```

Pentru a defini o funcție putem folosi mai multe metode. Putem trata cazurile diferite folosind `if`, ecuații (șabloane), cazuri etc.

De exemplu, pentru a defini funcția `fact` care calculează factorialul unui număr dat, putem folosi următoarele variante:

```
fact :: Integer -> Integer
```

- `Definiție folosind if`

  ```
  fact n = if n == 0 then 1 else n * fact(n-1)
  ```

- `Definiție folosind ecuații (șabloane)`

  ```
  fact 0 = 1
  fact n = n * fact(n-1)
  ```

- `Definiție folosind cazuri`

```
fact n
  | n == 0     = 1
  | otherwise = n * fact(n-1)
```

Funcția *semn* o putem defini matematic astfel:

$$semn\ n = \begin{cases} -1, & \textbf{dacă n} < \textbf{0} \\ 0, & \textbf{dacă n} = \textbf{0} \\ 1, & \textbf{altfel} \end{cases}$$

În Haskell, condițiile devin `gărzi`:

```
semn n
  | n < 0   = -1
  | n == 0  =  0
  | otherwise  = 1
```

Sau versiunea cu șabloane:

```
semn :: Integer -> Integer
semn 0 = 0
semn x
  | x > 0      = 1
  | otherwise = -1
```

În scrierea cu șabloane

- variabilele și valorile din partea stângă a semnului = sunt *șabloane*;
- când funcția este apelată se încearcă potrivirea parametrilor actuali cu șabloanele, ecuațiile fiind încercate *în ordinea scrierii*;
- în definiția factorialului, 0 și n sunt șabloane: 0 se va potrivi numai cu el însuși, iar n se va potrivi cu orice valoare de tip `Integer`.

3.6 Fișiere sursă

Fișierele sursă sunt fișiere cu extensia `.hs`, pe care le puteți edita cu un editor la alegerea voastră. Deschideți fișierul `lab1.hs` care conține următorul cod:

```
myInt =
    55555555555555555555555555555555555555555555555555555555555
double :: Integer -> Integer
double x = x+x
```

Fără a încărca fișierul, încercați să calculați double myInt:

ghci> double myInt

Observați mesajele de eroare. Acum încărcați fișierul folosind comanda `load (:l)` și încercați din nou să calculați `double myInt`:

```
ghci> :l lab1.hs
[1 of 1] Compiling Main             ( lab1.hs, interpreted )
Ok, 1 module loaded.
*Main> double myInt

*Main> double 2000
```

Modificați fișierul adăugând o funcție `triple`. Dacă fișierul este deja încărcat, puteți să îl reîncărcați folosind comanda `reload (:r)`.

Puteți reveni în `ghci` folosind `:m -`

```
ghci> :l lab1.hs
[1 of 1] Compiling Main             ( lab1.hs, interpreted )
Ok, 1 module loaded.
*Main> :m - Main
ghci>
```

3.7 Operatori

Operatorii sunt funcții cu două argumente, pot fi apelați folosind notația infix, pot fi definiți folosind numai "simboluri" (ex: *!*) - caz în care în definiția tipului operatorul este scris între paranteze.

- Operatori predefiniți

```
(||) :: Bool -> Bool -> Bool
(:) :: a -> [a] -> [a]
(+) :: Num a => a -> a -> a
```

- Operatori definiți de utilizator

```
(&&&) :: Bool -> Bool -> Bool  -- atentie la paranteze
True &&& b = b
False &&& _ = False
```

Putem defini și funcții ca operatori.

```
ghci> mod 5 2
1
ghci> 5 `mod` 2
1
```

Operatorii care sunt definiți în formă infix, sunt apelați în formă prefix folosind paranteze

```
2 + 3 == (+) 2 3
```

Operatorii care sunt definiți în formă prefix, sunt apelați în formă infix folosind ` ` *(backtick)*

```
mod 5 2 == 5 `mod` 2

elem :: a -> [a] -> Bool
ghci> 1 `elem` [1,2,3]
True
```

Precedență și asociativitate:

```
ghci> 3+5*4:[6]++8-2+3:[2]==[23,6,9,2]||True==False
True
```

Precedence	Left associative	Non-associative	Right associative
9	!!		.
8			^, ^^, **
7	*, /, `div`, `mod`, `rem`, `quot`		
6	+, −		
5			:, ++
4		==, /=, <, <=, >, >=, `elem`, `notElem`	
3			&&
2			||
1	>>, >>=		
0			$, $!, `seq`

Asociativitate:

Operatorul - asociativ la stânga

$5 - 2 - 1 == (5 - 2) - 1$ $-- /= 5 - (2 - 1)$

Operatorul : asociativ la dreapta

$5 : 2 : [] == 5 : (2 : [])$

Operatorul ++ asociativ la dreapta

```
(++) :: [a] -> [a] -> [a]
[] ++ ys = ys
(x:xs) ++ ys = x:(xs ++ ys)
```

$l1 ++ l2 ++ l3 ++ l4 ++ l5 == l1 ++ (l2 ++ (l3 ++ (l4 ++ l5)))$

Pentru a seta asociativitatea unui operator vom folosi următoarea notație:

```
infix[l|r] NUMAR <operatori-separati-prin-virgula>
```

- **infix** - neasociativ
- **infixr** - asociativ la dreapta
- **infixl** - asociativ la stânga
- NUMAR - precedența (între 0 și 9)

Pentru a descoperi asociativitatea unui operator se poate folosi comanda :i în GHCI
Exemple:

```
infix  4  ==, /=, <, <=, >=, >

infixr 3  &&
(&&) :: Bool -> Bool -> Bool

infixr 2  ||
(||) :: Bool -> Bool -> Bool

infixl 9 !!
(!!) :: [a] -> Int -> a

infixl 7 `div`
div :: Integral a => a -> a -> a
```

3.8 Exerciții

Exercițiul 3.3 Alegeți varianta corectă!

1. Cum se comentează o linie în Haskell?
   a) $--$
   b) $/**/$
   c) //
   d) !
2. Ce valoare are x în x = **let** x = 3 **in** x $*$ 5?
   a) 3
   b) 15
   c) 20
   d) Eroare
3. Ce valoare are x în x = **let** x = 3, y = 4 **in** x $*$ y
   a) 3
   b) 4
   c) 12
   d) eroare

4. Ce tip are o funcție f care are două argumente, primul argument de tip Char, iar al doilea argument de tip Bool, și întoarce un rezultat de tip Bool?

   a) f : **Char** −> **Bool** −> **Bool**

   b) f :: **Bool** −> **Char** −> **Bool**

   c) f :: **Char** −> **Bool** −> **Bool**

   d) nu se poate defini

5. Ce tip are expresia [**True**, 'a', "FP"]?

   a) (**Bool**, **Char**, **Char**)

   b) eroare

   c)[**Bool**, **Char**, [**Char**]]

   d)[**Bool**, **Char**, **Char**]

6. Ce tip are expresia (**True**, 'a', "FP")

   a) eroare

   b) (**Bool**, **Char**, **Char**)

   c) (**Bool**, **Char**, [**Char**])

   d) [**Bool**, **Char** [**Char**]]

7. Ce tip are o funcție f care are două argumente, o funcție de la Char la Bool și, respectiv, un Char, și întoarce un Bool?

   a) nu se poate defini

   b) f : (**Char** −> **Bool**) −> **Char** −> **Bool**

   c) f :: **Char** −> **Bool** −> **Char** −> **Bool**

   d) f :: (**Char** −> **Bool**) −> **Char** −> **Bool**

8. Ce valoare are f 3 în f 5 = **let** x = 3 **in** x + x?

   a) 6

   b) 5

   c) excepție (nu se potrivește niciun caz din definiția lui f)

   d) 10

9. Ce valoare are f 5 în f x = **let** x = 3 ; y = 4 **in** x + y?

   a) 9

   b) 7

   c) 5

   d) eroare

Exercițiul 3.4

Să se scrie următoarele funcții:

a) funcție cu 2 parametri care calculează suma pătratelor celor două numere;

b) funcție cu un parametru ce întoarce mesajul "par" dacă parametrul este par și "impar" altfel;

c) funcție care verifică dacă primul parametru este mai mare decât dublul celui de-al doilea parametru.

3.9 Secțiuni ("operator sections")

Aplicarea parțială a funcțiilor matematice:

Fie $f : A \times B \to C$ o funcție. În mod uzual scriem $f(a, b) = c$ unde $a \in A$, $b \in B$ și $c \in C$.

Pentru $a \in A$ și $b \in B$ (arbitrare, fixate) definim

$f_a : B \to C$, $f_a(b) = c$ dacă și numai dacă $f(a, b) = c$,

$f^b : A \to C$, $f^b(a) = c$ dacă și numai dacă $f(a, b) = c$.

Funcțiile f_a și f_b se obțin prin `aplicarea parțială` a funcției f.

Matematic, secțiunile corespund aplicării parțiale a unei funcții.

Secțiunile operatorului binar `op` sunt `(op e)` și `(e op)`.

Exemple:

- secțiunile lui `++` sunt `(++ e)` și `(e ++)`

```
ghci> :t (++ " world!")
(++ " world!") :: [Char] -> [Char]
ghci> (++ " world!") "Hello" -- atentie la paranteze
"Hello world!"
ghci> :t (++ " world!") "Hello"
(++ " world!") "Hello" :: [Char]
ghci> ++ " world!" "Hello"
    error

ghci> :t ("Hello" ++)
("Hello" ++) :: [Char] -> [Char]
ghci> ("Hello" ++) " world!"
"Hello world!"
ghci> :t ("Hello" ++) " world!"
("Hello" ++) " world!" :: [Char]
```

- secțiunile lui `<->` sunt `(<-> e)` și `(e <->)`, unde

```
ghci> let x <-> y = x-y+1  -- definit de utilizator
ghci> :t (<-> 3)
(<-> 3) :: Num a => a -> a
ghci> (<-> 3) 4
2
```

- secțiunile operatorului `(:)`

```
ghci> (2:)[1,2]
[2,1,2]
ghci> (:[1,2]) 3
[3,1,2]
```

- secțiunile operatorului `(+)`

```
ghci> f = (+2)
ghci> :t f
f :: Num a => a -> a
ghci> f 7
9
ghci> :t (+2)
(+2) :: Num a => a -> a
ghci> (+2) 7
9
```

- secțiunile operatorului `(-)`

```
ghci> (subtract 1) 11
10
ghci> (11 -) 1
10
ghci> :t (subtract 1)
(subtract 1) :: Num a => a -> a
ghci> :t (11 -)
(11 -) :: Num a => a -> a

ghci> map (subtract 1) [1,2,3,4]
[0,1,2,3]
```

Atenție!

```
ghci> :t -1   -- aici " - " nu este operator de sectiune
-1 :: Num a => a
ghci> -1
-1
ghci> :t (-1)
(-1) :: Num a => a
```

- secțiunile operatorului (^)

```
ghci> (^2) 3
9
ghci> (2^) 3
8
ghci> :t (2^)
(2^) :: (Integral b, Num a) => b -> a
ghci> :t (^2)
(^2) :: Num a => a -> a
ghci> ((-3)^) 2
9
```

Exercițiul 3.5

a) Scrieți o funcție cu 2 parametri care calculează suma pătratelor celor două numere folosind secțiuni.

```
f = (^2)   -- f x = x^2
g x y = f x + f y

ghci> g 2 3
13
```

b) Scrieți o funcție cu 3 parametri care verifică dacă se poate forma un triunghi dreptunghic având lungimile laturilor date de cei 3 parametri. Rezolvați acest exercițiu folosind secțiuni.

c) Scrieți o funcție poly care are patru argumente de tip Double, a,b,c,x și calculează $a * x^2 + b * x + c$. Scrieți și signatura funcției (poly :: ceva).

d) Scrieți o funcție eeny care întoarce "eeny" pentru input par și "meeny" pentru input impar. Hint: puteti folosi funcția even (puteți căuta pe `https://hoogle.haskell.org/`).

```
eeny :: Integer -> String
eeny = undefined
```

e) Scrieți o funcție fizzbuzz care întoarce "Fizz" pentru numerele divizibile cu 3, "Buzz" pentru numerele divizibile cu 5 și "FizzBuzz" pentru numerele divizibile cu ambele. Pentru orice alt număr se întoarce șirul vid. Pentru a calcula modulo a două numere puteți folosi funcția mod. Să se scrie această funcție în 2 moduri: folosind if și folosind gărzi (condiții).

4 Recursivitate

Una dintre diferențele dintre programarea declarativă și cea imperativă este modalitatea de abordare a problemei iterării: în timp ce in programarea imperativă acesta este rezolvată prin bucle (while, for, ...), în programarea declarativă rezolvarea iterării se face prin conceptul de recursie.
Un avantaj al recursiei față de bucle este acela că usurează sarcina de scriere și verificare a corectitudinii programelor prin raționamente de tip inductiv: construiește rezultatul pe baza rezultatelor unor subprobleme mai simple (aceeași problemă, dar pe o dimensiune mai mică a datelor).
Un foarte simplu exemplu de recursie este acela al calculării unui element de index dat din secvența numerelor Fibonacci, definită recursiv de:

$$F_n = \begin{cases} n & \text{dacă } n \in \{0, 1\} \\ F_{n-1} + F_{n-2} & \text{dacă } n > 1 \end{cases}$$

Putem transcrie această definiție direct in Haskell:

```
fibonacciCazuri :: Integer -> Integer
fibonacciCazuri n
    | n < 2     = n
    | otherwise = fibonacciCazuri (n - 1) + fibonacciCazuri (n -
      2)
```

Alternativ, putem folosi o definiție în stil ecuațional (cu șabloane):

```
fibonacciEcuational :: Integer -> Integer
fibonacciEcuational 0 = 0
fibonacciEcuational 1 = 1
fibonacciEcuational n = fibonacciEcuational (n - 1) +
   fibonacciEcuational (n - 2)
```

Fibonacci liniar O problemă cu definiția de mai sus este aceea că este timpul ei de execuție este exponențial. Motivul este acela că rezultatul este compus din rezultatele a 2 subprobleme de mărime aproximativ egală cu cea inițială.

Dar, deoarece recursia depinde doar de precedentele 2 valori, o putem simplifica cu ajutorul unei funcții care calculează recursiv perechea (F_{n-1}, F_n).

Exercițiul 4.1

Completați definiția funcției fibonacciPereche

Observație 1. Folosiți principiul de inducție: ne bazăm pe faptul ca fibonacciPereche (n−1) va calcula perechea (F_{n-2}, F_{n-1}) și o folosim pe aceasta pentru a calcula perechea (F_{n-1}, F_n).

Observație 2. Recursia este liniară *doar dacă* expresia care reprezintă apelul recursiv apare o singură dată. Folosiți **let**, **case** sau **where** pentru a vă asigura de acest lucru.

```
fibonacciLiniar :: Integer -> Integer
fibonacciLiniar 0 = 0
```

```
fibonacciLiniar n = snd (fibonacciPereche n)
  where
    fibonacciPereche :: Integer -> (Integer, Integer)
    fibonacciPereche 1 = (0, 1)
    fibonacciPereche n = undefined
```

Exercițiul 4.2

Numerele tribonacci sunt definite de ecuația

$$T_n = \begin{cases} 1 & \text{dacă } n = 1 \\ 1 & \text{dacă } n = 2 \\ 2 & \text{dacă } n = 3 \\ T_{n-1} + T_{n-2} + T_{n-3} & \text{dacă } n > 3 \end{cases}$$

Să se implementeze functia tribonacci atât cu cazuri cât și ecuațional.

```
tribonacci :: Integer -> Integer
tribonacci = undefined
```

Exercițiul 4.3

Să se scrie o funcție care calculează coeficienții binomiali, folosind recursivitate. Aceștia sunt determinați folosind urmatoarele ecuații.

B(n,k) = B(n-1,k) + B(n-1,k-1)
B(n,0) = 1
B(0,k) = 0

```
binomial :: Integer -> Integer -> Integer
binomial = undefined
```

Exercițiul 4.4

Să se implementeze următoarele funcții recursive, scriind la fiecare si signatura:

1. sumN - suma numerelor de la 1 la n;
2. putere x y - ridică pe x la puterea y;
3. inmultire - înmulțire prin adunări repetate
4. cifre - suma cifrelor unui număr

5 Funcții anonime (Lambda Expresii)

În Haskell funcțiile pot avea ca argument alte funcții. O modalitate de a transmite ca parametru funcții este utilizarea lambda expresiilor (funcții anonime).

Sintaxa seamană cu sintaxa lambda expresiilor din logică.

```
\ x1 x2 ..xn -> expresie

ghci> (\x -> x + 1) 3
4
ghci> inc = \x -> x + 1
ghci> inc 3
4
ghci> :t inc
inc :: Num a => a -> a

ghci> add = \x y -> x + y
ghci> add 4 6
10
ghci> :t add
add :: Num a => a -> a -> a

ghci> aplic = \f x -> f x
ghci> :t aplic
aplic :: (t1 -> t2) -> t1 -> t2

ghci> :t aplic add
aplic add :: Num t1 => t1 -> t1 -> t1
ghci> aplic add 2 3
5
ghci> :t aplic inc
aplic inc :: Num t2 => t2 -> t2
ghci> aplic inc 3
4

ghci> map (\x -> x+1) [1,2,3,4]
[2,3,4,5]
```

6 Compunerea funcțiilor — operatorul .

Matematic

Date fiind $f : A \to B$ și $g : B \to C$, compunerea lor, notată $g \circ f : A \to C$, este dată de formula

$$(g \circ f)(x) = g(f(x))$$

În Haskell.

```
(.) :: (b -> c) -> (a -> b) -> (a -> c)
(g . f) x = g (f x)
```

Exemplu

```
g :: Int -> Bool
g x = x > 0
```

```
f :: Int -> Int
f x = x + 1

ghci> :t (g.f)
(g.f) :: Int -> Bool

ghci> (g.f) 0
True
ghci> (g.f.f.f) (-2)
True
```

Exercițiul 6.1

Definiți 2 funcții care să calculeze dublul și triplul unui număr și găsiți toate modalitățile de compunere a acestor funcții.

```
h,t :: Int -> Int
```

7 Liste

Orice listă poate fi scrisă folosind doar constructorul `(:)` și lista vidă `[]`.

```
[1,2,3] == 1 : (2 : (3 : [])) == 1 : 2 : 3 : []

"abcd" == ['a','b','c','d'] == 'a':('b':('c':('d':[]))) == 'a':'b
    ':'c':'d':[]
```

Definiție recursivă. O **listă** este

- `vidă`, notată `[]`;

  sau

- `compusă`, notată `x:xs`, dintr-un un element `x` numit **capul listei** *(head)* și o listă `xs` numită **coada listei** *(tail)*.

Se pot defini `intervale și progresii artimetice`, dând regula progresiei între primii doi termeni din listă.

```
interval = ['c'..'e']          -- ['c','d','e']
progresie = [20,17..1]         -- [20,17,14,11,8,5,2]
progresie' = [2.0,2.5..4.0]    -- [2.0,2.5,3.0,3.5,4.0]
```

`Operații`

- !! - folosit pentru a găsi elementul de pe o anumită poziție din listă.
- ++ - concatenează două liste.
- : - concatenează un element la o listă.

- `head` - determină primul element dintr-o listă
- `tail` - determină lista fără primul element.
- `init` - determină lista fără ultimul element.
- `last` - determină ultimul element dintr-o listă.
- `length` - determină lungimea unei liste.
- `null` - determină dacă o listă este vidă.
- `take n l` - extrage primele n elemente dintr-o listă l.
- `drop n l` - șterge primele n elemente dintr-o listă l.
- `maximum/minimum` - retunează cel mai mare/mic element dintr-o listă.
- `sum/product` - retunează suma/produsul elementelor dintr-o listă.
- `elem e l` - determină dacă elementul e aparține listei l.

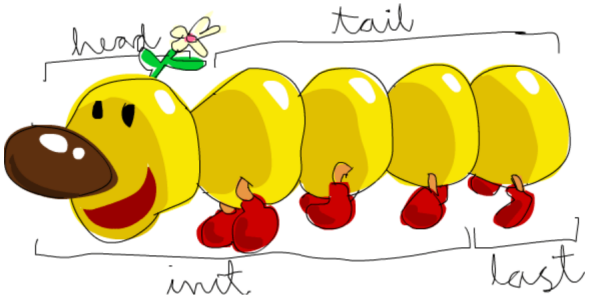
```
ghci> [1,2,3] !! 2
3
ghci> "abcd" !! 0
'a'
ghci> [1,2] ++ [3]
[1,2,3]
ghci> import Data.List

ghci> 5:[1,2,3,4,5]
[5,1,2,3,4,5]
ghci> 'P':"rogramareFunctionala"
"ProgramareFunctionala"
ghci> "P"++"rogramareFunctionala"
"ProgramareFunctionala"

ghci> b = [[1,2,3,4],[5,3,3,3],[1,2,2,3,4],[1,2,3]]
ghci> b
[[1,2,3,4],[5,3,3,3],[1,2,2,3,4],[1,2,3]]
ghci> b ++ [[1,1,1,1]]
[[1,2,3,4],[5,3,3,3],[1,2,2,3,4],[1,2,3],[1,1,1,1]]
ghci> [6,6,6]:b
[[6,6,6],[1,2,3,4],[5,3,3,3],[1,2,2,3,4],[1,2,3]]
ghci> b !! 2
[1,2,2,3,4]
ghci> b !! 2 !! 3
3
```

```
ghci> head "ProgramareFunctionala"
'P'
ghci> tail "ProgramareFunctionala"
"rogramareFunctionala"

ghci> init "ProgramareFunctionala"
"ProgramareFunctional"
ghci> last "ProgramareFunctionala"
'a'
```




```
ghci> head []
*** Exception: ghci.head: empty list
ghci> length "ProgramareFunctionala"
21

ghci> null [1,2,3]
False
ghci> null []
True

ghci> reverse "ProgramareFunctionala"
"alanoitcnuFeramargorP"

ghci> take 4 "ProgramareFunctionala"
"Prog"
ghci> drop 10 "ProgramareFunctionala"
"Functionala "

ghci> minimum [8,4,2,1,5,6]
1
ghci> maximum [1,9,2,3,4]
9
ghci> sum [8,4,2,1,4,5]
24
ghci> product [8,4,2,1,4,5]
1280
ghci> elem 8 [8,4,2,1,4,5]
```

```
True
ghci> elem 9 [8,4,2,1,4,5]
False
ghci> 5 `elem` [8,4,2,1,4,5]
True
ghci> 6 `elem` [8,4,2,1,4,5]
False
```

Șabloane (patterns) pentru liste

Listele sunt construite folosind constructorii (:) și []

```
[1,2,3] == 1:[2,3]  -- == 1:2:[3] == 1:2:3:[]
```

Observați:

```
ghci> let x:y = [1,2,3]
ghci> x
1
ghci> y
[2,3]
```

Ce s-a întâmplat?

- `x:y` este un șablon pentru liste
- potrivirea dintre `x:y` și `[1,2,3]` a avut ca efect:
  - "deconstrucția" valorii [1,2,3] în 1:[2,3]
  - legarea lui x la 1 și a lui y la [2,3]

Definiții folosind șabloane

Atenție! `șabloanele sunt definite folosind constructori.` De exemplu, operația de concatenare pe liste este

```
(++) :: [a] -> [a] -> [a]
```

Însă, [x] ++ [1] = [2,1] nu va avea ca efect legarea lui x la 2.

Încercând să evaluăm x vom obține un mesaj de eroare:

```
ghci> [x] ++ [1] = [2,1]
ghci> x
```

error: Variable not in scope: x

În Haskell șabloanele sunt `liniare`, adică o variabilă apare cel mult odată. Șabloane în care o variabilă apare de mai multe ori provoacă mesaje de eroare:

```
ghci> x:y:[1] = [2,2,1]
ghci> x
2
ghci> y
2
```

```
ghci> x:x:[1] = [2,2,1]
```

error: Multiple declarations of 'x'

```
ttail (x:x:t) = t
```

```
foo x x = x^2
```

error: Conflicting definitions for x

O soluție este folosirea gărzilor:

```
ttail (x:y:t)
 | (x==y) = t
 | otherwise = ...

foo x y
 | (x == y) = x^2
 | otherwise = ...
```

7.1 Recursivitate pe Liste

Recursivitatea pe liste se bazeaza pe definiția inductivă a listelor, prezentată mai sus. Funcțiile care parcurg recursiv listele vor folosi șabloanele pentru listă:

- lista vidă - [ ]
- listă compusă - $x : xs$

Exemplu: Dată fiind o listă de numere întregi, să se scrie o funcție semiPareRec care elimină numerele impare și le înjumătățește pe cele pare.

```
-- semiPareRec [0,2,1,7,8,56,17,18] == [0,1,4,28,9]
```

Prima implementare propusă este realizabilă in orice limbaj, folosind testul **null**, și „destructorii" **head** și **tail**.

```
semiPareRecDestr :: [Int] -> [Int]
semiPareRecDestr l
  | null l    = l
  | even h    = h `div` 2 : t'
  | otherwise = t'
  where
    h = head l
    t = tail l
    t' = semiPareRecDestr t
```

A doua implementare (preferată) folosește șabloane peste constructorul de listă : pentru a descompune lista:

```
semiPareRec :: [Int] -> [Int]
semiPareRec [] = []
semiPareRec (h:t)
  | even h    = h `div` 2 : t'
  | otherwise = t'
      where t' = semiPareRec t
```

Exemplu: Dată fiind o listă să se determine lista inversată.

```
reverse2 [ ] = [ ]
reverse2 (x:xs) = (reverse2 xs) ++ [x]
  -- x:xs se potriveste cu liste nevide
```

Exercițiul 7.1 Să se scrie următoarele funcții folosind recursivitate:

a) myreplicate - pentru un întreg n și o valoare v întoarce lista de lungime n ce are doar elemente egale cu v. Să se scrie și prototipul funcției.

b) sumImp - pentru o listă de numere întregi, calculează suma valorilor impare. Să se scrie și prototipul funcției.

c) totalLen - pentru o listă de șiruri de caractere, calculează suma lungimilor șirurilor care încep cu caracterul 'A'.

d) Să se scrie o funcție 'nrVocale' care pentru o listă de șiruri de caractere, calculează numărul total de vocale ce apar în cuvintele palindrom. Pentru a verifica dacă un șir e palindrom, puteți folosi funcția 'reverse', iar pentru a căuta un element într-o listă puteți folosi funcția 'elem'. Puteți defini oricâte funcții auxiliare.

Ex.: nrVocale ["sos", "civic", "palton", "desen", "aerisirea"] = 9

e) Să se scrie o funcție care primește ca parametru un număr și o listă de întregi, și adaugă elementul dat după fiecare element par din listă. Să se scrie și prototipul funcției.

Ex.: f 3 [1,2,3,4,5,6] = [1,2,3,3,4,3,5,6,3]

7.2 Liste definite prin selecție (descrieri de liste)

Haskell permite definirea unei liste prin selectarea și transformarea elementelor din alte liste sursă, folosind o sintaxă asemănătoare definirii mulțimilor matematice:

Matematic: $\{x | x \in \mathbb{N}, p(x)\}$, unde $p(x)$ este o proprietate cerută pentru x.

Haskell: [x | x <− [0..], p x]

Sintaxa poate fi dată de următoarea schemă:
[expresie | selectori , legari , filtrari], unde:

- `selectori`: una sau mai multe construcții de forma **pattern <− elista** (separate prin virgulă), unde `elista` este o expresie reprezentând o listă, iar `pattern` este un șablon pentru elementele listei `elista`.
- `legari`: zero sau mai multe expresii de forma **let pattern = expresie** folosind la legarea corespunzătoare a variabilelor din pattern cu valoarea expresie.

- `filtrari`: zero sau mai multe expresii de tip `Bool` (separate prin virgulă) folosite la păstrarea instanțelor selectate pentru care condiția e adevărată.
- `expresie`: expresie descriind elementele listei rezultat.

Exemple `[E(x)| x <- [x1,...,xn], P(x)]`

```
ghci> xs = [0..10]
ghci> [x | x <- xs, even x]
[0,2,4,6,8,10]

ghci> [x | x <- [1..10], x > 2 && x <= 9]
[3,4,5,6,7,8,9]

ghci> [x | x <- [1..10], x > 5 || odd x]
[1,3,5,6,7,8,9,10]

ghci> xs = [0..6]
ghci> [(x,y) | x <- xs, y <- xs,  x + y == 10]
[(4,6),(5,5),(6,4)]
```

Folosirea lui `let` pentru declarații locale:

```
ghci> [(i,j) | i <- [1..2], let k = 2 * i, j <- [1..k]]
[(1,1),(1,2),(2,1),(2,2),(2,3),(2,4)]
```

Pentru $i = 1$:

- $k = 2$
  - $j \leftarrow [1..2] : (1,1), (1,2)$

Pentru $i = 2$:

- $k = 4$
  - $j \leftarrow [1..4] : (2,1), (2,2), (2,3), (2,4)$

Iată cum arată o posibilă implementare a funcției `semiPare` folosind selecție pe liste:

```
semiPareComp :: [Int] -> [Int]
semiPareComp l = [ x `div` 2 | x <- l, even x ]
```

Exercițiul 7.2 Încercați să găsiți valoarea expresiilor de mai jos și verificați răspunsul găsit de voi în interpretor:

```
[ x^2 |x <- [1..10], x `rem` 3 == 2]

[(x,y)| x<- [1..5], y <- [x..(x+2)]]

[(x,y)| x<-[1..3], let k = x^2, y <- [1..k]]

[ x | x <- "Facultatea de Matematica si Informatica",
  elem x ['A'..'Z']]

[[x..y]| x <- [1..5], y <- [1..5], x < y]
```

Exercițiul 7.3

Scrieți o funcție care date fiind limita inferioară și cea superioară (întregi) a unui interval închis și o listă de numere întregi, calculează lista numerelor din listă care aparțin intervalului. De exemplu:

inInterval 5 10 [1..15] == [5,6,7,8,9,10]
inInterval 5 10 [1,3,5,2,8,−1] = [5,8]

a) Folosiți doar recursie. Denumiți funcția **inIntervalRec**.

b) Folosiți selecție pe liste. Denumiți funcția **inIntervalComp**.

Exercițiul 7.4

Scrieți o funcție care numără câte numere strict pozitive sunt într-o listă dată ca argument.

De exemplu:

pozitive [0,1,−3,−2,8,−1,6] == 3

a) Folosiți doar recursie. Denumiți funcția **pozitiveRec**.

b) Folosiți selecție pe liste. Denumiți funcția **pozitiveComp**.

Exercițiul 7.5

Scrieți o funcție care calculează produsul tuturor cifrelor care apar în șirul de caractere dat ca intrare. Dacă nu sunt cifre în șir, răspunsul funcției trebuie să fie 1. De exemplu:

```
multDigits "The time is 4:25" == 40
multDigits "No digits here!" == 1
```

a) Folosiți doar recursie. Denumiți funcția **multDigitsRec**.

b) Folosiți selecție pe liste. Denumiți funcția **multDigitsComp**.

Indicație: Veți avea nevoie de funcția **isDigit** care verifică dacă un caracter e cifră și funcția **digitToInt** care transformă un caracter în cifră. Cele 2 funcții se află în pachetul **Data.Char**.

Exercițiul 7.6

Scrieți o funcție care pentru o listă de valori (reprezentând niște prețuri) aplică un discount de 25% acelor valori și pstrează în listă valorile reduse care sunt mai mici decât 200. De exemplu:

```
discount [150, 300, 250, 200, 450, 100] == [112.5, 187.5, 150.0,
    75.0 ]
```

- Folosiți doar recursie. Denumiți funcția **discountRec**.
- Folosiți descrieri de liste. Denumiți funcția **discountComp**.

7.3 Prelucrarea listelor folosind funcțiile: Map, Filter, Zip și Fold

7.3.1 Map

Funcția `map` are ca argumente o funcție de tip $a \to b$ și o listă de elemente de tip a, rezultatul fiind lista elementelor de tip b obținute prin aplicarea funcției date pe fiecare element de tip a. O posibilă implementare a funcției ar fi:

```
map :: (a -> b) -> [a] -> [b]
map f xs =[f x | x <- xs]
```

Exemple:

```
ghci> map (* 3) [1,3,4]
[3,9,12]
ghci> map (++ "abc") ["acesta", "este", "un", "test"]
["acestaabc","esteabc","unabc","testabc"]
```

Încercați să găsiți valoarea expresiilor de mai jos și verificați răspunsul găsit de voi în interpretor:

```
map (\x -> 2 * x) [1..10]
map (1 `elem`) [[2,3],[1,2]]
map (`elem` [2,3]) [1,3,4,5]
```

7.3.2 Filter

Funcția `filter` are ca argument o proprietate si o listă de elemente, rezultatul fiind lista elementelor care verifică acea proprietate. O posibilă implementare a funcției ar fi:

```
filter :: (a -> Bool) -> [a] -> [a]
filter p xs = [x | x <- xs, p x]
```

Exemple:

```
ghci> filter (>2) [3,1,4,2,5]
[3,4,5]
ghci> filter odd [3,1,4,2,5]
[3,1,5]
```

Exercițiul 7.7

Scrieți o funcție **prop** care dată fiind o valoare verifică dacă aceasta este impară și mai mică de 50. Folosind funcția **prop** definți o nouă funcție **lista_impare_50** care primește o listă de întregi și returnează o listă doar cu elementele ce îndeplinesc proprietatea **prop**.

```
ghci> lista_impare_50 [1..100]
[1,3,5,7,9,11,13,15,17,19,21,23,25,27,29,31,33,35,37,39,41,43,45,47,49]
```

Exercițiul 7.8

Scrieți o funcție **lista_impare_n** care primește o listă de întregi și un număr n și returnează o listă doar cu dublul elementelor care sunt impare și mai mici decât valoarea n.

```
ghci> lista_impare_n [1..100] 20
[2,6,10,14,18,22,26,30,34,38]
```

Exercițiul 7.9 Scrieți o funcție **sumaImpare** care pentru o listă de numere întregi calculează suma valorilor impare.

```
ghci> sumaImpare [1..5]
9
ghci> sumaImpare [1..6]
9
ghci> sumaImpare [1..10]
25
```

7.3.3 Zip

Funcția **zip** are ca argumente două liste și va determina lista de perechi formate din combinarea elementelor de pe aceleași poziții din cele două liste.

```
ghci> :t zip
zip :: [a] -> [b] -> [(a, b)]

ghci> ys = ['A'..'E']
ghci> zip [1..] ys
[(1,'A'),(2,'B'),(3,'C'),(4,'D'),(5,'E')]

ghci> xs = ['A'..'Z']
ghci> [x | (i,x) <- [1..] `zip` xs, even i]
"BDFHJLNPRTVXZ"
```

Observați diferența!

```
ghci> zip [1..3] ['A'..'D']
[(1,'A'),(2,'B'),(3,'C')]

ghci> [(x,y) | x <- [1..3], y <- ['A'..'D']]
[(1,'A'),(1,'B'),(1,'C'),(1,'D'),
(2,'A'),(2,'B'),(2,'C'),(2,'D'),
(3,'A'),(3,'B'),(3,'C'),(3,'D')]
```

Exercițiul 7.10

Scrieți o funcție care dată fiind o listă de numere calculează lista pozițiilor elementelor impare din lista originală. De exemplu:

pozitiiElemImpare [0,1,−3,−2,8,−1,6,1] == [1,2,5,7]

a) Folosiți doar recursie. Denumiți funcția **pozitiiElemImpareRec**. Indicație: folosiți o funcție ajutătoare, cu un argument în plus reprezentând poziția curentă din listă.

b) Folosiți selecție pe liste. Denumiți funcția **pozitiiElemImpareComp**. Indicație: folosiți funcția zip pentru a asocia poziții elementelor listei.

Rezolvați următoarele exerciții folosind map, fillter și zip (fără recursivitate sau selecție). Pentru fiecare funcție scrieți și prototipul acesteia.

Exercițiul 7.11

Scrieți o funcție generică `firstEl` care are ca argument o listă de perechi de tip (a,b) și întoarce lista primelor elementelor din fiecare pereche (puteți să folosiți și funcția **fst** predefinită):

firstEl [('a',3),('b',2), ('c',1)] == "abc"

Exercițiul 7.12

Scrieți funcția `sumList` care are ca argument o listă de liste de valori Int și întoarce lista sumelor elementelor din fiecare listă (suma elementelor unei liste de întregi se calculează cu funcția **sum** predefinită):

sumList [[1,3], [2,4,5], [], [1,3,5,6]] == [4,11,0,15]

Exercițiul 7.13

Scrieți o funcție `prel2` care are ca argument o listă de Int și întoarce o listă în care elementele pare sunt înjumătățite, iar cele impare sunt dublate:

prel2 [2,4,5,6] == [1,2,10,3]

Exercițiul 7.14

Scrieți o funcție **list_elem_sir** care primește ca argument un caracter și o listă de șiruri, rezultatul fiind lista șirurilor care conțin caracterul respectiv (folosiți funcția **elem** predefinită).

```
ghci> list_elem_sir 's' ["sos", "palton", "desen", "aerisire"]
["sos","desen","aerisire"]
```

Exercițiul 7.15

Scrieți o funcție **patrate_impare** care primește ca argument o listă de întregi și întoarce lista pătratelor numerelor impare.

```
ghci> patrate_impare [3..10]
[9,25,49,81]
```

Exercițiul 7.16

Scrieți o funcție **lista_patrate_poz_imp** care primește ca argument o listă de întregi și întoarce lista pătratelor numerelor din poziții impare. Pentru a avea acces la poziția elementelor folosiți zip.

Exercițiul 7.17

Scrieți o funcție **numaiVocale** care primește ca argument o listă de șiruri de caractere și întoarce lista obținută prin eliminarea consoanelor din fiecare șir.

```
ghci> numaiVocale ["laboratorul", "ProgrAmare", "FuncTIONala"]
["aoaou","oAae","uIOaa"]
```

Exercițiul 7.18

Definiți recursiv funcțiile `mymap` și `myfilter` cu aceeași funcționalitate ca și funcțiile predefinite.

7.3.4 Fold

`Funcțiile foldr și foldl`

Definiție: Date fiind o funcție de actualizare a valorii calculate cu un element curent, o valoare inițială și o listă, calculați valoarea obținută prin aplicarea repetată a funcției de actualizare fiecărui element din listă.

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr op z [a1,a2,a3,...,an] =
  a1`op`(a2`op`(a3`op`(...(an`op`z)...)))


foldl :: (b -> a -> b) -> b -> [a] -> b
foldl op z [a1,a2,a3,...,an] =
 (...(((z`op`a1)`op`a2)`op`a3) ...)`op` an
```

Definițiile recursive:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f i []     = i
foldr f i (x:xs) = f x (foldr f i xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl h i []     = i
foldl h i (x:xs) = foldl h (h i x) xs
```

Exemple:

```
foldr (+) 0 [1, 2, 3] == 1 + (2 + (3 + 0))

foldr (*) 1 [1, 2, 3] == 1 * (2 * (3 * 1))

ghci> foldr (++) [] ["Ana ", "are ", "mere."]
 "Ana " ++ ("are " ++ ("mere." ++ []))

ghci> foldr (++) "a" ["ana","are","mere"]
"anaaremerea"

ghci> foldl (++) "a" ["ana","are","mere"]
"aanaaremere"
```

– Suma elementelor dintr-o listă –

Definiți o funcție care dată fiind o listă de numere întregi calculează suma elementelor din listă.

Soluție cu foldr

```
sum = foldr (+) 0
```

Cu **foldr**, elementele sunt procesate de la dreapta la stânga:
sum $[x_1, \ldots, x_n] = (x_1 + (x_2 + \ldots (x_n + 0) \ldots)$

Soluție în care elementele sunt procesate de la stânga la dreapta.

```
sum :: [Int] -> Int
sum xs = suml xs 0
          where
              suml [] n = n
              suml (x:xs) n = suml xs (n+x)
```

Elementele sunt procesate de la stânga la dreapta:
suml $[x_1, \ldots, x_n]$ $0 = (\ldots (0 + x_1) + x_2) + \ldots x_n)$

Soluție cu foldl

```
sum :: [Int] -> Int
sum xs = foldl (+) 0 xs
```

– Suma pătratelor numerelor pozitive –

```
f :: [Int] -> Int
f xs = sum (squares (positives xs))
```

```
f :: [Int] -> Int
f xs = sum [ x*x | x <- xs, x > 0 ]
```

```
f :: [Int] -> Int
f []                = 0
f (x:xs)| x > 0     = (x*x) + f xs
        | otherwise = f xs
```

```
f :: [Int] -> Int
f xs = foldr (+) 0 (map sqr (filter pos xs))
  where
    sqr x = x * x
    pos x = x > 0
```

```
f :: [Int] -> Int
f xs = foldr (+) 0
         (map ( \x -> x * x)
            (filter ( \x -> x > 0) xs))

f :: [Int] -> Int
f xs = foldr (+) 0 (map (^2) (filter (>0) xs))

f :: [Int] -> Int
f = foldr (+) 0 . map (^2) . filter (>0)
```

– Evaluare leneșă. Liste infinite –

Putem folosi funcțile **map** și **filter** pe liste infinite:

```
ghci> inf = map (+10) [1..] -- inf nu este evaluat
ghci> take 3 inf
[11,12,13]
```

Limbajul Haskell folosește implicit evaluarea leneșă.

- expresiile sunt evaluate numai când este nevoie de valoarea lor
- expresiile nu sunt evaluate total, elementele care nu sunt folosite rămân neevaluate
- o expresie este evaluată o singură dată.

În exemplul de mai sus, este acceptată definiția lui inf, fără a fi evaluată. Când expresia **take** 3 inf este evaluată, numai primele 3 elemente ale lui inf sunt calculate, restul rămânând neevaluate.

Să considerăm următoarele funcții:

```
primes = sieve [2..]
sieve (p:ps) = p : sieve [ x | x <- ps, mod x p /= 0 ]
```

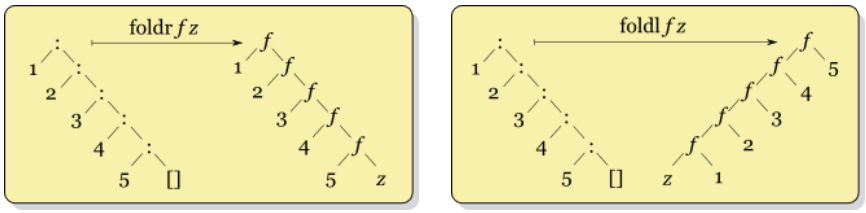
Intuitiv, evaluarea leneșă funcționează astfel:

```
sieve [2..] -->

2 : sieve [ x | x <- [3..], mod x 2 /= 0 ] -->

2 : sieve (3 : [ x | x <- [4..], mod x 2 /= 0 ])  -->

2 : 3 : sieve ([ y | y <-
                    [x | x <- [4..], mod x 2 /= 0 ],
                     mod y 3 /= 0 ])
--> ...
```



https://en.wikipedia.org/wiki/Fold_(higher-order_function)

- **foldr** poate fi folosită pe liste infinite (în anumite cazuri),
- **foldl nu poate fi folosită pe liste infinite niciodată.**

```
ghci> foldr (*) 0 [1..]
*** Exception: stack overflow

ghci> take 3 (foldr (\x xs-> (x+1):xs) [] [1..])
[2,3,4]
-- foldr a functionat pe o lista infinita

ghci> take 3 (foldl (\xs x-> (x+1):xs) [] [1..])
-- expresia se calculeaza la infinit
```

Exercițiul 7.19

(a) Scrieți o funcție recursivă care calculează produsul numerelor dintr-o listă.

```
produsRec :: [Integer] -> Integer
produsRec = undefined
```

(b) Scrieți o funcție echivalentă care folosește 'foldr' în locul recursiei.

```
produsFold :: [Integer] -> Integer
produsFold = undefined
```

Exercițiul 7.20

(a) Scrieți o funcție recursivă care verifică faptul că toate elementele dintr-o listă sunt 'True'.

```
andRec :: [Bool] -> Bool
andRec = undefined
```

(b) Scrieți o funcție echivalentă care folosește 'foldr' în locul recursiei.

```
andFold :: [Bool] -> Bool
andFold = undefined
```

Exercițiul 7.21

(a) Scrieți o funcție recursivă care concatenează o listă de liste.

```
concatRec :: [[a]] -> [a]
concatRec = undefined
```

(b) Scrieți o funcție echivalentă care folosește 'foldr' în locul recursiei.

```
concatFold :: [[a]] -> [a]
concatFold = undefined
```

Exercițiul 7.22

(a) Scrieți o funcție care elimină un caracter din șir de caractere.

```
rmChar :: Char -> String -> String
rmChar = undefined
```

(b) Scrieți o funcție recursivă care elimină toate caracterele din al doilea argument care se găsesc în primul argument, folosind 'rmChar'.

```
rmCharsRec :: String -> String -> String
rmCharsRec = undefined

test_rmchars :: Bool
test_rmchars = rmCharsRec ['a'..'l'] "fotbal" == "ot"
```

(c) Scrieți o funcție echivalentă cu cea de la (b) care folosește 'foldr' în locul recursiei.

```
rmCharsFold :: String -> String -> String
rmCharsFold = undefined
```

Exercițiul 7.23 Scrieți o funcție care returnează suma pătratelor elementelor impare dintr-o listă dată ca parametru. Hint: folosiți filter, map, foldr.

```
sumImp :: [Int] -> Int
sumImp = undefined
```

8 Tipuri de date algebrice

8.1 Definire tipuri de date

Tipuri sumă

- În Haskell tipul **Bool** este definit astfel:

  ```
  data Bool = False | True
  ```

  Unde:

  - **Bool** este constructor de tip
  - **False** și **True** sunt constructori de date

- În mod similar putem defini

  ```
  data Season = Spring | Summer
              | Autumn | Winter
  ```

  Unde:

  - Season este constructor de tip
  - Spring, Summer, Autumn și Winter sunt constructori de date

Bool și Season sunt tipuri de date **sumă**, adică sunt definite prin **enumerarea alternativelor**.

```
data Bool = False | True
```

- Operațiile se definesc prin "pattern matching":

  ```
  not :: Bool -> Bool
  not False = True
  not True  = False

  (&&), (||) :: Bool -> Bool -> Bool
  False && q = False
  True  && q = q
  False || q = q
  True  || q = True
  ```

Putem să definim pentru tipul sumă anotimpuri următoarele operații:

```
data Season = Spring | Summer
            | Autumn | Winter

succesor Spring = Summer
succesor Summer = Autumn
```

```
succesor  Autumn = Winter
succesor  Winter = Spring


showSeason Spring = "Primavara"
showSeason Summer = "Vara"
showSeason Autumn = "Toamna"
showSeason Winter = "Iarna"
```

Tipuri produs

- Să definim un tip de date care să aibă ca valori "punctele" cu două coordonate de tipuri oarecare:

  ```
  data Point a b = Pt a b
  ```

  Unde:

  - Point este constructor de tip
  - Pt este constructor de date

- Pentru a accesa componentele, definim **proiecțiile**:

  ```
  pr1 :: Point a b -> a
  pr1 (Pt x _) = x
  pr2 :: Point a b -> b
  pr2 (Pt _ y) = y
  ```

Point este un tip de date **produs**, definit prin **combinarea** tipurilor a și b.

```
data Point a b = Pt a b

ghci> :t (Pt 1 "c")
(Pt 1 "c") :: Num a => Point a [Char]

ghci> :t Pt
Pt :: a -> b -> Point a b
-- constructorul de date este operatie

ghci> :t (Pt 1)
(Pt 1) :: Num a => b -> Point a b
```

- Se pot defini operații:

  ```
  pointFlip :: Point a b -> Point b a
  pointFlip (Pt x y) = Pt y x

  ghci> pr1 $ pointFlip (Pt 1 "a")
  "a"
  ghci> pr2 $ pointFlip (Pt 1 "a")
  1
  ```

Tipuri de date definite recursiv

- Declarația listelor ca tip de date algebric

```
data List a = Nil
            | Cons a (List a)
```

- Se pot defini operații

```
append :: List a -> List a -> List a
append Nil ys           = ys
append (Cons x xs) ys = Cons x (append xs ys)
```

Tipurile de date algebrice se definesc folosind "operațiile" sumă și produs.

Forma generală

$$\begin{aligned} data\ Typename\ = \ & Cons_1\ t_{11} \ldots t_{1k_1} \\ & | Cons_2\ t_{21} \ldots t_{2k_2} \\ & | \ldots \\ & | Cons_n\ t_{n1} \ldots t_{nk_n} \end{aligned}$$

unde $k_1, \ldots, k_n \geq 0$

- Se pot folosi tipuri sumă și tipuri produs.
- Se pot defini tipuri parametrizate.
- Se pot folosi definiții recursive.

Atenție! Alternativele trebuie să conțină **constructori**.

data StrInt = **String** | **Int** este greșit

data StrInt = VS **String** | VI **Int** este **corect**
[VI 1, VS "abc", VI 34, VI 0, VS "xyz"] :: [StrInt]

Exemple de tipuri de date algebrice:

```
data Bool = False | True

data Season = Winter | Spring | Summer | Fall

data Shape = Circle Float | Rectangle Float Float

data Maybe a = Nothing | Just a

data Pair a b = Pair a b
  -- constructorul de tip si cel de date pot sa coincida
```

```
data Nat = Zero | Succ Nat

data Exp = Lit Int | Add Exp Exp | Mul Exp Exp

data List a = Nil | Cons a (List a)

data Tree a = Empty | Leaf a | Branch (Tree a) (Tree a)
```

Constructori simboluri

```
data List a = Nil
            | Cons a (List a)
```

Declarație ca tip de date algebric cu simboluri

```
data List a = Nil
            | a ::: List a
      deriving (Show)

infixr 5 :::

ghci> 1:::2:::Nil
1 ::: (2 ::: Nil)

ghci> :t 1:::2:::Nil
1:::2:::Nil :: Num a => List a
```

Liste și tupluri

- Liste

  ```
  data [a] = [] | a : [a]
  ```

  Constructorii listelor sunt [] și : unde

  ```
  [] :: [a]
  (:) :: a -> [a] -> [a]
  ```

- Tupluri

  ```
  data (a,b) = (a,b)
  data (a,b,c) = (a,b,c)
    ...                ...
  ```

  Nu există o declarație generică pentru tupluri, fiecare declarație de mai sus definește tuplul de lungimea corespunzătoare, iar constructorii pentru fiecare tip în parte sunt:

  ```
  (,) :: a -> b -> (a,b)
  (,,) :: a -> b -> c -> (a,b,c)
    ...
  ```

Utilizarea type

Cu **type** se pot redenumi tipuri deja existente.

```
type FirstName  = String
type LastName   = String
type Age        = Int
type Height     = Float
type Phone = String
```

Exemplu de un tip de date ce reține informații personale:

```
data Person = Person FirstName LastName Age Height Phone

    firstName :: Person -> String
    firstName (Person firstname _ _ _ _) = firstname

    lastName :: Person -> String
    lastName (Person _ lastname _ _ _) = lastname

    age :: Person -> Int
    age (Person _ _ age _ _) = age

    height :: Person -> Float
    height (Person _ _ _ height _ ) = height

    phoneNumber :: Person -> String
    phoneNumber (Person _ _ _ _ number) = number

ghci> let ionel = Person "Ion" "Ionescu" 20 175.2  "0712334567"

ghci> firstName ionel
"Ion"

ghci> height ionel
175.2

ghci> phoneNumber ionel
"0712334567"
```

Date personale ca înregistrări:

```
data Person = Person { firstName :: String
                     , lastName :: String
                     , age :: Int
                     , height :: Float
                     , phoneNumber :: String
                     }
```

- Putem folosi atât forma algebrică, cât și cea de înregistrare

```
ionel = Person "Ion" "Ionescu" 20 175.2 "0712334567"

gigel = Person { firstName = "Gheorghe"
               , lastName="Georgescu"
               , age = 30, height = 192.3
               , phoneNumber = "0798765432"
               }
```

- Putem folosi și pattern-matching
- Proiecțiile sunt definite automat; sintaxă specializată pentru actualizări

```
nextYear :: Person -> Person
nextYear person = person { age = age person + 1 }
```

```
*Main> nextYear ionel
```

No instance for (Show Person) arising from a use of 'print'

Deși toate definițiile sunt corecte, o valoare de tip Person nu poate fi afișată deoarece nu este instanță a clasei **Show**.

Exercițiul 8.1 În continuare vom scrie câteva funcții definite folosind tipul de date 'Fruct':

```
data Fruct
   = Mar String Bool
   | Portocala String Int
```

O expresie de tipul 'Fruct' este fie un 'Mar String Bool' sau o 'Portocala String Int'. Vom folosi un 'String' pentru a indica soiul de mere sau portocale, un 'Bool' pentru a indica dacă mărul are viermi și un 'Int' pentru a exprima numărul de felii dintr-o portocală. De exemplu:

```
ionatanFaraVierme = Mar "Ionatan" False
goldenCuVierme = Mar "Golden Delicious" True
portocalaSicilia10 = Portocala "Sanguinello" 10
listaFructe = [Mar "Ionatan" False,
               Portocala "Sanguinello" 10,
               Portocala "Valencia" 22,
               Mar "Golden Delicious" True,
               Portocala "Sanguinello" 15,
               Portocala "Moro" 12,
               Portocala "Tarocco" 3,
               Portocala "Moro" 12,
               Portocala "Valencia" 2,
               Mar "Golden Delicious" False,
               Mar "Golden" False,
               Mar "Golden" True]
```

a) Scrieți o funcție

```
ePortocalaDeSicilia :: Fruct -> Bool
ePortocalaDeSicilia = undefined
```

care indică dacă un fruct este o portocală de Sicilia sau nu. Soiurile de portocale din Sicilia sunt 'Tarocco', 'Moro' și 'Sanguinello'. De exemplu:

```
test_ePortocalaDeSicilia1 =
  ePortocalaDeSicilia (Portocala "Moro" 12) == True
test_ePortocalaDeSicilia2 =
    ePortocalaDeSicilia (Mar "Ionatan" True) == False
```

b) Scrieți o funcție

```
nrFeliiSicilia :: [Fruct] -> Int
nrFeliiSicilia = undefined

test_nrFeliiSicilia = nrFeliiSicilia listaFructe == 52
```

care calculează numărul total de felii ale portocalelor de Sicilia dintr-o listă de fructe.

c) Scrieți o funcție

```
nrMereViermi :: [Fruct] -> Int
nrMereViermi = undefined

test_nrMereViermi = nrMereViermi listaFructe == 2
```

care calculează numărul de mere care au viermi dintr-o listă de fructe.

8.2 Instanțe ale claselor de tipuri

Derivare automată pentru tipuri algebrice

Am definit tipuri de date noi:

- Tipul Anotimpuri

```
data Season = Spring | Summer | Autumn | Winter
              deriving (Eq, Ord, Show)

!!! Fara derivare automata
ghci> Summer>Autumn
<interactive>:17:7: error:
    * No instance for (Ord Season) arising from a use of `>'

ghci> Summer == Autumn

<interactive>:18:8: error:
    * No instance for (Eq Season) arising from a use of `=='
```

```
!!!Cu derivare automata
ghci> Summer > Winter
False
ghci> Summer > Spring
True
```

- Tipul Puncte

```
data Point a b = Pt a b
                 deriving (Eq, Ord, Show)

!!! Fara derivare automata
ghci> Pt 1 2 < Pt 2 3
<interactive>:2:8: error:
    * No instance for (Ord (Point Integer Integer))
        arising from a use of `<'

!!!Cu derivare automata
ghci> Pt 1 2 < Pt 2 3
True
ghci> Pt 1 2 < Pt 1 3
True
ghci> Pt 1 2 < Pt 1 1
False
ghci> Pt 1 2 < Pt 7 1
True
ghci> Pt 1 "a" < Pt 7 "b"
True
ghci> Pt 1 "a" < Pt 1 "b"
True
ghci> Pt 1 "a" < Pt 1 "A"
False
```

Cum putem să le facem instanțe ale claselor **Eq**, **Ord**, **Show**?

Putem să le facem explicit sau să folosim derivarea automată.

Atenție!
Derivarea automată poate fi folosită numai pentru unele clase predefinite.

Derivare automata vs Instanțiere explictă

- O **clasă de tipuri** este determinată de o mulțime de funcții.

```
class Eq a where
  (==) :: a -> a -> Bool
  (/=) :: a -> a -> Bool
  -- minimum definition: (==)
  x /= y = not (x == y)
```

- Tipurile care aparțin clasei sunt **instanțe** ale clasei.

- Instanțierea prin derivare automată:

```
data Point a b = Pt a b
                 deriving Eq
ghci> Pt 1 2 == Pt 1 3
False
```

- Instanțiere explicită:

```
instance Eq a => Eq (Point a b) where
        (==) (Pt x1 y1) (Pt x2 y2) = (x1 == x2)

ghci> Pt 1 2 == Pt 1 3
True
```

Exemplu de instanțiere explicită pentru tipul Anotimpuri

```
data Season = Spring | Summer | Autumn | Winter

instance Eq Season where
  Spring == Spring = True
  Summer == Summer = True
  Autumn == Autumn = True
  Winter == Winter = True
  _      ==     _  = False


instance Show Season where
  show Spring = "Primavara"
  show Summer = "Vara"
  show Autumn = "Toamna"
  show Winter = "Iarna"

!!!Cu derivare automata  -- deriving Show
ghci> Spring
Spring


!!!Cu instantiere explicita
ghci> Spring
Primavara
```

Pentru instanța de Show putem să ne folosim de funcția **showSeason** definită la începutul acestui capitol.

Exemplu de instanțiere explicită pentru tipul List

```
data List a = Nil
            | a ::: List a
          deriving (Show)

infixr 5 :::
```

- Putem defini operații:

```
(+++) :: List a -> List a -> List a
infixr 5 +++
Nil +++ ys       = ys
(x ::: xs) +++ ys = x ::: (xs +++ ys)
```

Comparați cu versiunea folosind notația predefinită:

```
(++) :: [a] -> [a] -> [a]
[] ++ ys       = ys
(x:xs) ++ ys = x : (xs ++ ys)
```

```
ghci> 1:::2:::3:::Nil +++ 1:::2:::3:::Nil
1 ::: (2 ::: (3 ::: (1 ::: (2 ::: (3 ::: Nil)))))
```

Definirea egalității și a reprezentării

```
eqList :: Eq a => List a -> List a -> Bool
eqList Nil Nil = True
eqList (x ::: xs) (y:::ys) = x == y && eqList xs ys
eqList _ _ = False

instance (Eq a) => Eq (List a) where
   (==) = eqList
```

```
ghci> 1:::Nil +++ 2:::Nil == 1:::2:::Nil
True
```

```
showList :: Show a => List a -> String
showList Nil = "Nil"
showList (x ::: xs) = show x ++ " ::: " ++ showList xs

instance (Show a) => Show (List a) where
   show = showList
```

```
ghci> 1:::2:::3:::Nil +++ 1:::2:::3:::Nil
1 ::: 2 ::: 3 ::: 1 ::: 2 ::: 3 ::: Nil
```

8.3 Parțialitate - tipul Maybe

```
data Maybe a = Nothing | Just a
```

Argumente opționale

```
power :: Maybe Int -> Int -> Int
power Nothing n    = 2 ^ n
power (Just m) n = m ^ n
```

Rezultate opționale

```
divide :: Int -> Int -> Maybe Int
divide n 0 = Nothing
divide n m = Just (n `div` m)
```

```
-- utilizare gresita
  wrong :: Int -> Int -> Int
  wrong n m = divide n m + 3

-- utlizare corecta
  right :: Int -> Int -> Int
  right n m = case divide n m of
                  Nothing -> 3
                  Just r -> r + 3
```

```
ghci> right 10 0
3
ghci> right 100 9
14
ghci> 100 `div` 9 => 11
ghci> fromMaybe 0 (Just 5) => 5
ghci> fromMaybe 0 Nothing => 0

right' :: Int -> Int -> Int
right' n m = (fromMaybe 0 (divide n m)) + 3
```

Exercițiul 8.2 Definiți o funcție **find** cu două argumente: un predicat și o listă de elemente. Funcția **find** găsește primul element din lista pentru care predicatul returnează True. Returnează Nothing dacă nu există un astfel de element.

```
find :: (a -> Bool) -> [a] -> Maybe a
find pred li = undefined
```

Exercițiul 8.3 Definiți o funcție **maybe_sqrt** care primește o listă de elemente de tipul Maybe Float și înlocuiește doar elementele pozitive cu radicalul lor, cele negative le dublează, iar restul de elemente rămân neschimbate.

Exercițiul 8.4

```
type NumeA = String
type Rasa = String
data Animal = Pisica NumeA | Caine NumeA Rasa
    deriving Show
```

a) Definiți o funcție

```
vorbeste :: Animal -> String
vorbeste = undefined
```

care întoarce '"Meow!"' pentru pisică și '"Woof!"' pentru câine.

b)Definiți o funcție

```
rasa :: Animal -> Maybe String
rasa = undefined
```

care întoarce rasa unui câine dat ca parametru sau 'Nothing' dacă parametrul este o pisică.

c) Definiți o funcție **findPisica** care primește un animal de tip Pisica și o listă de animale alcătuită din câini și pisici și returnează un Maybe String. Dacă pisica dată ca argument se găsește în lista de animale, atunci, folosind funcția **vorbeste**, veți returna 'Just "Meow!"', altfel returnați 'Nothing'.

d) Definiți o funcție **findNume** care primessțe un nume de animal și o listă de animale și returnează primul animal care deține acest nume. Dacă numele nu se găsește în listă, atunci returnează Nothing.

8.4 Variante - tipul Either

Either A B (A sau B)

```
data Either a b = Left a | Right b

mylist :: [Either Int String]
mylist = [Left 4, Left 1, Right "hello", Left 2,
          Right " ", Right "world", Left 17]
```

Definiți o funcție care calculează suma elementelor întregi.

```
addints    :: [Either Int String] -> Int
addints    []                = 0
addints    (Left n : xs)     = n + addints xs
addints    (Right s : xs) = addints xs


addints' :: [Either Int String] -> Int
addints' xs = sum [n | Left n <- xs]
```

```
data Either a b = Left a | Right b


mylist :: [Either Int String]
mylist = [Left 4, Left 1, Right "hello", Left 2,
          Right " ", Right "world", Left 17]
```

Definiți o funcție care întoarce concatenarea elementelor de tip **String**.

```
addstrs  :: [Either Int String] -> String
addstrs  []              = ""
addstrs  (Left n : xs)   = addstrs xs
addstrs  (Right s : xs) = s ++ addstrs xs


addstrs' :: [Either Int String] -> String
addstrs' xs = concat [s | Right s <- xs]
```

Exercițiul 8.5 Definiți o funcție **safeDiv** care divide două argumente, iar în caz de eroare întoarce Left cu un mesaj corespunzător de eroare.

Exercițiul 8.6

```
type Agenda = [(String, String)]
lista_Agenda :: Agenda
lista_Agenda = [("ana","0722-222-222"),
                ("ion", "0722-222-111"),
                ("ionel", "0722-222-333")]

type E_Present = String
type E_Absent = String
```

Definiți o funcție **checkAgenda** care primește un nume și o agendă și întoarce Right "Gasit" dacă a gasit o persoană cu același nume și întoarce Left "Nu este in agenda" în caz contrar. Folosiți fold.

```
checkAgenda :: String -> Agenda -> Either E_Absent E_Present
checkAgenda nume lista = undefined
```

Exercițiul 8.7

```
data User = User
  { name :: String ,
    picture :: Either String Int
  }
    deriving Show
```

Un User are un nume și o poză. Dacă poza a fost încărcată, atunci se va seta Right ⟨*idPoza*⟩, altfel, se va seta Left "no foto".

a) Definiți o funcție **list_pic** ce primește o listă de useri și întoarce o listă cu informațiile despre pozele de profil ale acestora.

```
list_users = [User "Ana" (Right 100),
 User "Ion" (Left "no foto"),
 User "Ionel" (Left "no foto")]

ghci> list_pic list_users
[Right 100,Left "no foto",Left "no foto"]
```

b) Definiți o funcție **have_pic** care primește o listă de useri și retunează o listă cu mesaje corespunzatoare în funcție de existența sau absența unei poze de profil.

```
ghci> have_pic list_users
[" are "," Nu are "," Nu are "]
```

Exercițiul 8.8

Se dau următoarele tipuri de date ce reprezintă matrice cu linii de lungimi diferite:

```
data Linie = L [Int]
    deriving Show
data Matrice = M [Linie]
   deriving Show
```

a) Definiți o funcție **verifica** care verifică dacă suma elementelor de pe fiecare linie este egală cu o valoare 'n'. Rezolvați cerința folosind 'foldr'.

```
verifica :: Matrice -> Int -> Bool
verifica = undefined

test_veri1 = verifica (M[L[1,2,3], L[4,5], L[2,3,6,8], L[8,5,3]])
   10 == False

test_verif2 = verifica (M[L[2,20,3], L[4,21], L[2,3,6,8,6], L
   [8,5,3,9]]) 25 == True
```

b) Definiți o funcție **doarPozN** care are ca parametru un element de tip 'Matrice' și un număr întreg 'n', și care verifică dacă toate liniile de lungime 'n' din matrice au numai elemente strict pozitive.

```
doarPozN :: Matrice -> Int -> Bool
doarPozN = undefined

testPoz1 =
doarPozN (M [L[1,2,3], L[4,5], L[2,3,6,8], L[8,5,3]]) 3 == True

testPoz2 =
doarPozN (M [L[1,2,-3], L[4,5], L[2,3,6,8], L[8,5,3]]) 3 == False
```

c) Definiți predicatul **corect** care verifică dacă toate liniile dintr-o matrice au aceeași lungime.

```
corect :: Matrice -> Bool
corect = undefined

testcorect1 = corect (M[L[1,2,3], L[4,5], L[2,3,6,8], L[8,5,3]])
   == False
testcorect2 = corect (M[L[1,2,3], L[4,5,8], L[3,6,8], L[8,5,3]])
   == True
```

d) Să se instanțieze clasa Eq pentru tipul Linie care îndeplinește următoarea prorietate: două linii sunt egale dacă au exact aceleași elemente.

e) Să se instanțieze clasa Eq pentru tipul Matrice care îndeplinește următoarea prorietate: două matrice sunt egale dacă au exact aceleași elemente(ca în matematică).

f) Cum modificăm această ultimă instanță dacă vrem să fie îndeplinită următoarea prorietate: două matrice sunt egale dacă au același număr de linii și colone (nu se va ține cont de elementele lor).

Exercițiul 8.9

Alegeți varianta corectă!

1. Cum putem defini lista [3,4,5,6]? Sunt mai multe variante corecte la acest exercitiu?

a) 3 : 4 : 5 : 6
b) 3 : 4 : 5 : 6 : []
c) [3 .. 6]
d) 3 : 4 : 5 : [6]

2. Ce obtinem dupa instructiunile?

```
ghci> xs = [1,2,3]

ghci> ys = [11,12]

ghci> zip xs ys
```

a) nu se poate aplica functia zip
b) [(1,11),(2,12)]
c) [1,2,3,11,12]
d) [(1,11),(1,12),(2,11),(2,12),(3,11),(3,12)]

3. Ce obtinem dupa instructiunile?

```
ghci> let natural = [0..]

ghci> natural !! 5
```

a) 6
b) 5
c) [0,1,2,3,4]
d) 4

4. Fie f x = x + x si g x = x * x. Ce valoare are expresia g . f $ 3?
a) 36
b) 18
c) eroare
d) 6

5. Ce obtinem dupa instructiunea ([1,2,3] ++) [4,5,6]?
a) eroare
b) [1,2,3,4,5,6]
c) [4,5,6,1,2,3]
d) "123456"

6. Ce obtinem dupa instructiunea (++ [1,2,3]) [4,5,6]?
a) eroare
b) [1,2,3,4,5,6]
c) [4,5,6,1,2,3]
d) "123456"

7. Ce obtinem dupa instructiunea reverse . take 3 [1 .. 10]?
a) [10,9,8]
b) [1,2,3]
c) eroare
d) [3,2,1]

8. Ce obtinem dupa instructiunea (reverse . take 3) [1 .. 10]?
a) [10,9,8]
b) [1,2,3]
c) eroare
d) [3,2,1]

9. Ce obtinem dupa instructiunea (take 3 . reverse) [1 .. 10]?
a) [10,9,8]
b) [1,2,3]
c) eroare
d) [3,2,1]

Exercițiul 8.10

Alegeți varianta corectă!
1. Ce se obtine dupa instructiunea **map** (+1) [1,2,3,4]?
a) nu se poate aplica
b) [2,3,4,5]
c) [4,3,2,1]
d) [2,3,4]

2. Ce se obtine dupa instructiunea **map** (1−) [1,2,3,4]?
a) nu se poate aplica
b) [2,3,4,5]
c) [0,1,2,3]

d) [0,-1,-2,-3]

3. Ce se obtine dupa instructiunea **map toUpper** "abcd"?
a) nu se poate aplica
b) "dcba"
c) "ABCD"
d) "Abcd"

4. Ce se obtine dupa instructiunea **length** . **filter** (== 'a') "abracadabra"?
a) 5
b) "brcdbr"
c) instructiune invalida
d) "aaaaa"

5. Ce se obtine dupa instructiunea (**length** . **filter** (== 'a')) "abracadabra"?
a) 5
b) "brcdbr"
c) instructiune invalida
d) "aaaaa"

6. Ce se intampla dupa instructiunea **filter** (\x -> (**rem** x 2) == 0) [1..10]?
a) [2,4,6,8,10]
b) [1,3,5,7,9]
c) 5
d) instructiune invalida

7. Ce se obtine dupa instructiunea **foldr** (++) ["woot","WOOT","woot"]?
a) "wootWOOTwoot"
b) instructiune invalida
c) ["woot","WOOT","woot"]
d) "woot,WOOT,woot"

8. Ce se obtine dupa urmatoarea instructiune **foldr** (&&) **True** [**False**,**True**]?
a) instructiune invalida
b) False
c) True
d) [True,False,True]

9. Ce se obtine dupa urmatoarea instructiune
foldr (\ x y -> **concat** ["(",x,"+",y,")"]) "0" ["1","2","3","4","5"]?
a) instructiune invalida
b) "(1+(2+(3+(4+(5+0)))))"
c) "1+2+3+4+5+0"
d) ["(","1","2","3","4","5",")"]

10. Ce se obtine dupa urmatoarea instructiune **foldl** (^) 2 [1..3] ?
a) 1
b) 64
c) instructiune invalida
d) 8

11. Ce se obtine dupa urmatoarea instructiune **foldr** (^) 2 [1..3] ?
a) 1
b) 64
c) instructiune invalida
d) 8

12. Ce se obtine dupa urmatoarea instructiune **foldr** (:) [] [1..3] ?
a) []
b) instructiune invalida
c) [1,2,3]
d) [3,2,1]

13. Ce se obtine dupa urmatoarea instructiune **foldl** (**flip** (:)) [] [1..3] ?
a) [1,2,3]
b) instructiune invalida
c) [3,2,1]
d) []

Exercițiul 8.11

```
Fie tipul de date:
data Doggies a =
       Husky a
     | Mastiff a
```

1. Ce este Doggies?
a) constructor de tip
b) constructor de date
c) tip de date produs
d) niciunul din raspunsurile de mai sus

2. Ce tip are Mastiff "Scooby Doo"?
a) Doggies
b) [Char]
c) Doggies [Char]
d) Doggies Mastiff

3. Ce tip are Husky (10 :: Integer)?
a) Doggies
b) Doggies Integer
c) Integer
d) Doggies Husky

Exercițiul 8.12

1. Clasa Eq
a) include toate tipurile din Haskell
b) coincide cu clasa Ord
c) face testarea egalitatii posibila
d) include doar tipuri numerice

2. Sa presupunem ca clasa de tipuri Ord are operatorul >. Ce tip are >?
a) **Ord** a =>a −> a −> **Bool**
b) **Ord** a =>**Int** −> **Bool**
c) **Ord** a =>a −> **Char**
d) **Ord** a =>**Char** −> [**Char**]

3. Ce puteti sa spuneti despre codul de mai jos?

```
data Mood = Blah
          | Woot deriving Show

settleDown x = if x == Woot
               then Blah
               else x
```

a) codul este corect

b) codul nu este corect deoarece nu exista o instanta a clasei Num pentru tipul Mood

c) codul nu este corect deoarece nu exista o instanta a clasei Ord pentru tipul Mood

d) codul nu este corect deoarece nu exista o instanta a clasei Eq pentru tipul Mood